



CSI247: Data Structures

Lecture 08 – Packages

Department of Computer Science

B. Gopolang (247-275)

Previous Lessons

- Time complexity of algorithm
 - Algorithm's growth rate
 - Big-O notation

Today ...

Learning Objectives

- By the end of this, lecture you must be able to:
 - Understand what **packages** are
 - Understand various Java's built in packages
 - Understand and differentiate the 2 categories of packages
 - Create a user-defined package
 - Create a small Java program using a user-defined packages

Introduction

Package: A group of related classes organized into a folder

- Advantages
 - Promotes writing better and maintainable code
 - Prevent possibility of name conflicts
- Two categories:
 - Built-in Packages (refer to Java API)
 - Come bundled with the language
 - User-defined Packages
 - Must be created by programmers

a) Built-in packages

- Java's API is a **huge** library of pre-written classes
 - Rem: class is a blue-print of objects belonging to it
 - Examples of classes:
 - Scanner: input (e.g. keyboard)
 - PrintWriter: file output
 - StringTokenizer: breaking a String into tokens
 - Each of the above belongs to some package
 - The API is divided into packages



Java defined core packages

- Java has core packages
 - Each serves a different purpose
 - **You have used these packages!**
- Examples include:
 - java.lang: for Java's fundamental classes, e.g. String, Math
 - java.util: for collections framework and utility classes.
e.g. Date, Random number generator, etc
 - java.io: for classes associated with file input and output
 - etc (See Java API documentation for more)

How have we been using these packages?

- Add **import** keyword before your class definition
- Syntax:

import *packageName.className;*

- E.g.
 - Import java.io.*;
- Note
 - **java.lang** package is imported by default in Java

Example: using a built-in package

- Remember Scanner class for reading from the keyboard?
- It is in the **java.util** package

How have we been importing Scanner class

- `import java.util.*;` `// import all classes in the package`
- `import java.util.Scanner;` `// import Scanner only`

Which Scanner methods have we used?

- `next()`, `nextInt()`, `nextLine()`, `hasNext()`, etc

b) User-defined packages

- We can create our user defined classes & package them
 - Create several user-defined classes
 - Bundle related classes together into a package
- Then import the package / relevant class(es) in our programs!

Steps?

- a) Create the necessary directory structure
 - src directory: java files
 - classes directory: class files and packages go here

b) Choose package name

- E.g. *pkg*
- Recommendation for package naming
 - Start package name with a lower case letter
 - Use Caps at the beginning of each new word

c) Create source files (.java) under src folder

d) Insert keyword **package** followed by package name at the top of every source file to include in the package

- Syntax

```
package package_name;
```

illustration

```
package pkg;  
public class A {  
    // data & methods  
}
```

- Save as A.java

```
package pkg;  
public class B {  
    // data & methods  
}
```

- Save as B.java

Note:

- Both classes have package called **pkg** inserted at the top
- They will be added to package **pkg**

d) Create the package

How?

- Compile each class to be included in the package, individually
javac -d classes/ -sourcepath src src/.../class_name;

Note:

- Compile from parent directory of src and classes folder
- -d: create directory with package_name indicated in source files
- classes: package_name destination (or .(dot) for current directory)
- Upon successful compilation:
 - **Classes** directory will have a sub-directory called **pkg**
 - **pkg** will have a class file for each compiled .java file

e) Using the package in a program:

- Insert **import** keyword (as 1st line), followed by *package_name.class_name* in Tester class

- Syntax

```
import package_name.class_name;
```

- Example

```
import pkg.A; // or import pkg.*; (for all classes)
```

```
public class pkgTester {
```

```
// ...
```

```
}
```

e) Testing if the package works properly

How?

- By executing the Tester class
 - Use this command:

```
java -classpath classes packagefolder.Tester_name;
```

Note:

- You must work from folder above classes folder

Summary

- Packages
 - Built-in packages
 - User-defined packages
 - Declaration
 - Using them

Next lesson

- Searching Arrays

Q & A



Thank you